

MessengerSlayer Code Style

Версия 1.0 — Официальное руководство по написанию кода (Перевод на русский язык)

1. Основные принципы (Core principles)

1. Одна зона ответственности — один класс — один файл.
2. Отдавайте предпочтение понятному и явному коду, а не «хитроумным» решениям.
3. Не внедряйте временные архитектурные решения, которые впоследствии планируется переписать.
4. Новый функционал должен расширять существующие уровни, а не обходить их.
5. Не допускайте циклических зависимостей в проекте.
6. Не размещайте бизнес-логику в слоях UI, транспорта и инфраструктуры персистентности (хранения данных).
7. Каждый архитектурный компонент, доступный извне, должен иметь четко определенную зону ответственности.
8. Избегайте избыточного проектирования (overengineering) и ненужных абстракций.
9. Пишите код, который можно объяснить другому разработчику без скрытых допущений.
10. Новые функции не должны нарушать существующую работоспособность и работу тестов.

2. Зоны ответственности проектов (Project responsibilities)

MessengerSlayer.Client

Обязанности: WPF, MVVM, Представления (Views), Модели представлений (ViewModels), Команды (Commands), Клиентские сервисы, Навигация, Состояние UI (UI state), Диспетчеризация UI-потока (UI thread dispatching), Оркестрация клиентского сетевого взаимодействия.

НЕ ДОЛЖЕН: обращаться напрямую к SQL, содержать серверную бизнес-логику, содержать репозитории базы данных, напрямую управлять низкоуровневым TCP из Views.

Предпочтительная цепочка вызовов (Flow):

View → **ViewModel** → **Client Service** → **Networking** → **Transport** → **Server**

MessengerSlayer.Contracts

Обязанности: Запросы (Requests), Ответы (Responses), DTO (объекты передачи данных), События (Events), Перечисления (Enums), Контракты протокола, Общие модели данных, независимые от транспорта.

НЕ ДОЛЖЕН: обращаться к базе данных, ссылаться на WPF, содержать серверную бизнес-логику, содержать конкретную реализацию сетевого взаимодействия.

MessengerSlayer.Transport

Обязанности: Stream (Потоки), TCP, TLS, Фрейминг (Framing), Бинарный макет пакета (Binary packet layout), Кодирование, Точные чтения из потока, Чтение/запись пакетов, Транспортные исключения.

НЕ ДОЛЖЕН знать про: Пользователей (Users), Чаты (Chats), Сообщения (Messages), SQL, WPF, Бизнес-правила.

MessengerSlayer.Security

Обязанности: Хеширование паролей, Криптографические примитивы, Управление ключами, Локальная защита ключей, DPAPI, Шифрование/расшифровка сообщений, Цифровые подписи, Проверка целостности, E2EE (Сквозное шифрование).

НЕ ДОЛЖЕН: содержать UI, содержать запросы к базе данных, содержать бизнес-логику чатов.

MessengerSlayer.Data

Обязанности: Entity Framework, ADO.NET, DbContext, Сущности базы данных (Database entities), Репозитории, Запросы, Транзакции, Миграции.

НЕ ДОЛЖЕН: ссылаться на WPF, содержать сетевые обработчики, содержать клиентскую логику.

MessengerSlayer.Server

Обязанности: Хостинг сервера, Жизненный цикл TCP-соединения, Сессии, Аутентификация, Авторизация, Маршрутизация запросов, Обработчики запросов (Request handlers), Серверные прикладные сервисы, Пользователи, Чаты, Обмен сообщениями, Файлы, Статус присутствия (Presence), Синхронизация, Оркестрация логирования.

Обработчики (Handlers) НЕ ДОЛЖНЫ содержать сырой SQL.

MessengerSlayer.Tests

Обязанности: Юнит-тесты, Интеграционные тесты, Сетевые тесты, Тесты безопасности, Тесты базы данных, Сквозные тесты (End-to-end / E2E), Симуляция сбоев, Регрессионные тесты.

3. Язык и наименования (Language and naming)

Все идентификаторы в коде должны быть на английском языке.

Используйте **PascalCase** для: классов, структур, перечислений (enums), методов, свойств, публичных полей, констант.

Используйте **camelCase** для: локальных переменных, параметров методов.

Используйте **_camelCase** для приватных полей. Имена интерфейсов должны начинаться с буквы **I**.

```
public interface IMessageService
{
}

public sealed class MessageService : IMessageService
{
    private readonly IMessageRepository _messageRepository;

    public MessageService(IMessageRepository messageRepository)
    {
        _messageRepository = messageRepository
            ?? throw new ArgumentNullException(nameof(messageRepository));
    }
}
```

Асинхронные методы должны оканчиваться на **Async** (например, **SendMessageAsync**).

Имена переменных и свойств типа **bool** должны читаться как вопросы (например: **IsConnected**, **IsAuthenticated**, **HasPrivateKey**, **CanReconnect**, **ShouldRetry**).

4. Правила оформления файлов (File rules)

- Один главный тип на файл. Имя файла должно совпадать с именем главного типа (например, `MessageService.cs`).
- Избегайте абстрактных имен файлов, таких как `Helpers.cs`, `Utils.cs`, `Managers.cs`, `Common.cs`, `Stuff.cs`.
- Приватные вспомогательные методы могут оставаться внутри того же класса, если они поддерживают его единственную ответственность.

5. Правила написания классов (Class rules)

- Используйте модификатор `sealed` для классов, которые не предназначены для наследования.
- Отдавайте предпочтение внедрению зависимостей через конструктор (Constructor Injection). Зависимости должны быть `readonly`.
- Не выполняйте тяжелую работу в конструкторах (не открывайте соединения, не выполняйте SQL, сетевые или криптографические операции).

6. Форматирование (Formatting)

- Используйте стиль фигурных скобок Allman (на отдельной строке). Всегда используйте фигурные скобки, даже для однострочных блоков.
- Используйте 4 пробела для отступов. Не используйте директиву `#region` по умолчанию.
- Избегайте избыточной вложенности. Отдавайте предпочтение ранним возвратам (Early returns).

7. Использование типов (Type usage)

- Отдавайте предпочтение явным типам, когда они улучшают читаемость (например: `Message message = repository.Get(id);`).
- Использование `var` допустимо только тогда, когда тип очевиден из правой части выражения (`var users = new List<User>();`).

8. Проверка на Null и валидация аргументов

Валидируйте аргументы публичных методов и конструкторов с помощью `ArgumentNullException` и `ArgumentException`. Не игнорируйте молча некорректные входные данные.

9. Исключения (Exceptions)

- Не используйте базовый `throw new Exception()`. Отдавайте предпочтение конкретным или доменным исключениям (например, `InvalidPacketFrameException`).
- Никогда не перехватывайте исключения впустую (не «проглатывайте» их в пустой `catch`).

10. Правила асинхронности (Async rules)

- Сетевые, файловые и операции ввода-вывода (I/O) базы данных должны быть асинхронными.
- Используйте `CancellationToken` самым последним параметром. Избегайте блокирующих вызовов `.Result` и `.Wait()`.
- В библиотечном и серверном коде использование `ConfigureAwait(false)` разрешено и предпочтительно.

11. Правила MVVM (MVVM rules)

- Views содержат только логику отображения и не обращаются к TCP, SQL, шифрованию или аутентификации.
- Используйте связывание команд `Command="{Binding SendMessageCommand}"` вместо обработчиков кликов в code-behind.

12. Поток в WPF (WPF threading)

Сетевые колбэки не должны напрямую изменять `ObservableCollection` из фоновых потоков. Используйте абстракцию вроде `IUiDispatcher`.

13. Контракты и JSON (Contracts and JSON)

- C# свойства — `PascalCase`; JSON свойства — `camelCase`.
- Сетевые временные метки должны использовать UTC (например, `CreatedAtUtc`).
- Перечисления (Enums) сериализуются как читаемые строки (`"messageType": "Text"`).
- Бинарные данные передаются через транспортный payload, а не Base64 в JSON.

14. Идентификаторы (IDs)

- Используйте явные наименования (`UserId`, `ChatId`, `MessageId`). Избегайте `Id1`, `DataId`.
- Клиентские ID обычно используют `Guid`; серверные ID в БД — `int` / `long`.
- Идентификатор отправителя всегда берется из аутентифицированной сессии сервера, клиент не может его подделать.

15. Сетевой протокол (Network protocol)

- Все запросы/ответы коррелируются с помощью `RequestId` / `CorrelationId`.
- Сервер валидирует размер фрейма, версию протокола, тип пакета и состояние авторизации.
- Используйте логику точного чтения из потока (exact-read). Параллельные записи в поток должны быть синхронизированы.

16. TLS и безопасность (TLS and security)

- TLS обязателен. Не создавайте валидаторы сертификатов, которые просто возвращают `true`.
- Не хардкодьте пароли и ключи. Никогда не логируйте пароли, токены, приватные ключи и E2EE-текст.
- Используйте проверенные криптографические библиотеки, не изобретайте свои алгоритмы.

17. Обработка паролей (Password handling)

Никогда не храните пароли в открытом виде или через простой MD5/SHA256. Используйте Argon2id или PBKDF2.

18. Доступ к данным (Data access)

- **Entity Framework:** CRUD домена, связи, пользователи, чаты, сессии, метаданные.
- **ADO.NET:** оптимизированное чтение истории, синхронизация, счетчики непрочитанных, bulk-операции.
- Всегда используйте параметризованный SQL. Не конкатенируйте строки.

19. Обработчики на стороне сервера (Server handlers)

Каждый тип запроса имеет отдельный файл обработчика (например, `LoginRequestHandler.cs`). Обработчики не содержат сырой SQL.

20. Прикладные сервисы / Прецеденты использования

Создавайте небольшие узкоспециализированные сервисы (`RegisterUserService`, `SendMessageService`), избегая гигантских «всемогущих» классов.

21. Логирование, комментарии и проектирование методов

- Логи должны быть структурированными (`Event=UserAuthenticated UserId=42`). Секреты не логируются.
- Комментарии объясняют **ПОЧЕМУ**, а не **ЧТО**.
- Один метод — одна логическая операция.

22. Правила тестирования и Git

- Тесты обязательны и следуют структуре `Arrange → Act → Assert`.
- Сообщения коммитов Git пишутся на английском языке и должны быть лаконичными и осмысленными.

23. Правила для ИИ при генерации кода

1. Строго следовать данному `CODESTYLE.md`.
2. Указывать статус файлов: `NEW`, `UPDATED`, `UNCHANGED`.
3. Предоставлять полные версии всех измененных файлов без частичных заплат.
4. Использовать английские термины Visual Studio.
5. Не обходить слои архитектуры и не изобретать собственную криптографию.

Золотые правила (Golden Rules)

1. Одна ответственность → один класс → один файл.
2. UI никогда не общается с TCP или SQL напрямую.
3. Transport знает байты и потоки, а не бизнес-логику.
4. Contracts содержат общие контракты данных, а не бизнес-поведение.
5. Серверные обработчики не содержат SQL.
6. Код безопасности никогда не логирует секреты.
7. Операции ввода-вывода (I/O) асинхронны, где это возможно.
8. Каждый запрос можно коррелировать с ответом.
9. Сетевые и серверные метки времени используются в UTC.
10. Тесты — неотъемлемая часть реализации.
11. Новые функции расширяют архитектуру, а не обходят её.
12. Стабильная работающая инфраструктура не переписывается без веской причины.
13. Если код трудно объяснить — упростите его перед коммитом.